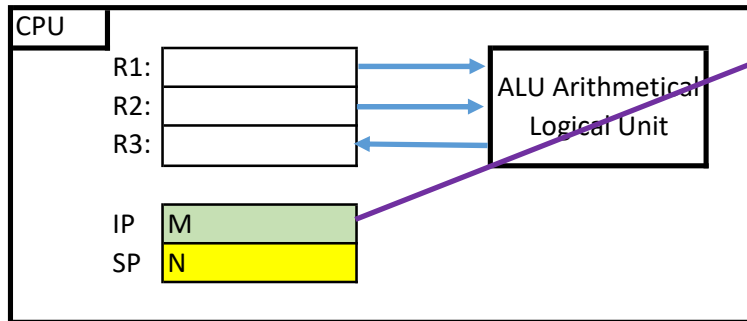


10. CPU_MEM_ADD: $v = v + 2;$

Addition of 2 to variable v: $v = v + 2;$ or $v += 2;$

1 IP points to current instruction: Add 2 to variable v

Adresse	0	1	...	Variable v	...	M	M+1	M+2			N-2	N-1	N
Memory:				123		Add	2	to v	...				



Description:

R1, R2, R3: General purpose register.

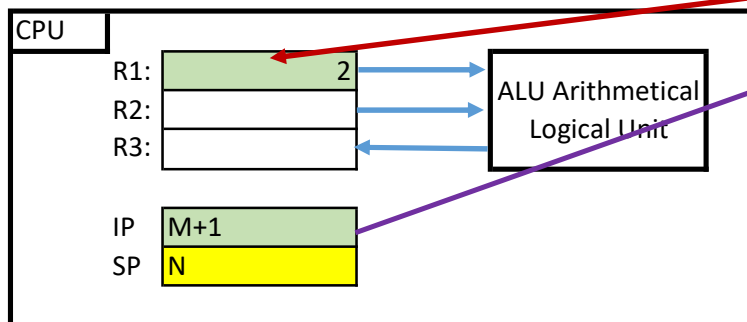
ALU: Does the maths and comparisons.

IP: Instruction pointer, used to run the programm ("Current instruction").

SP: Stack Pointer, not used here.

2 Execution: 2 is transferred to register R1

Adresse	0	1	...	Variable v	...	M	M+1	M+2			N-2	N-1	N
Memory:				123		Add	2	to v	...				

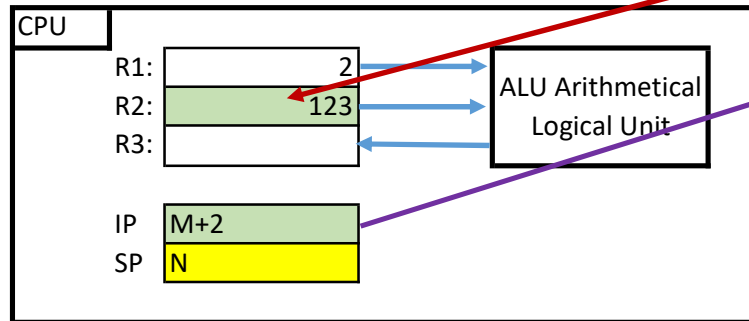


The value 2 is moved to register R1.

Any value in R1 is overwritten.

3 Execution: Content of v is transferred to register R2

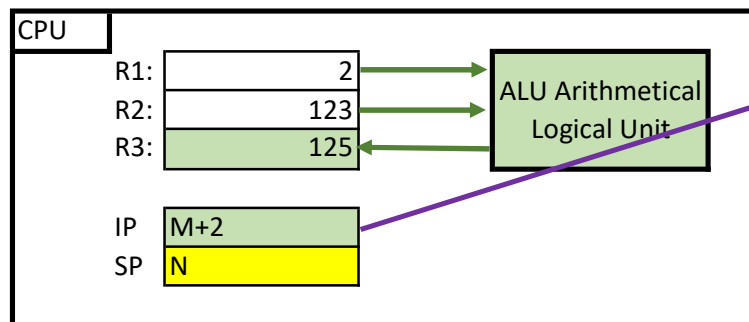
Adresse	0	1	...	Variable v	...	M	M+1	M+2			N-2	N-1	N
Memory:				123		Add	2	to v	...				



The value of variable v is moved to register R2.
Any value in R2 is overwritten.

4 Execution: Add R1 to R2 and store in R3

Adresse	0	1	...	Variable v	...	M	M+1	M+2			N-2	N-1	N
Memory:				123		Add	2	to v	...				

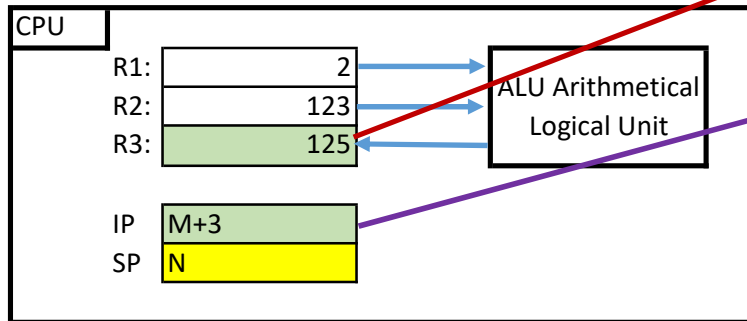


ALU is triggered to do the maths.
Result is stored in R3.
Any value in R3 is overwritten.

10. CPU_MEM_ADD: $v = v + 2;$

5 Execution: Store R3 back to variable v and carry on (IP++)

Adresse	0	1	...	Variable v	...	M	M+1	M+2	M+3		N-2	N-1	N
Memory:				125		Add	2	to v	...				



After finishing the instruction add, incl. moving result back to v,
the contents of R1, R2 and R3 are "garbage".

Information processing works like a manufacturing plant(!)

The CPU, or CPUs (can be more than 1) are the machines.

The CPUs form a "production line".

The CPU "cores" are part of the "production line" too.

The memory is the warehouse ("Lager").

The back-and-forth between CPU and memory (machines and warehouse) is called in manufacturing "Intra-logistic".

To skip the back-and-forth between CPU and memory, for heavily used variables one can recommend in C/C++ register:

register int i;

Example:

```
for( register int i = 0; i < N; i++) {  
    /* do something */  
}
```

Putting i in a register saves ***a lot*** of traffic between CPU and memory, precisely: $2 \cdot N$ traffic is avoided.

"register" is only a recommendation. Compilers may choose to ignore it, especially

if the CPU does not have enough registers or these are used otherwise.

Modern compilers are supposed to automatically identify situations where a variable should be put in a register.

Nevertheless, **register** recommendations have never killed performance :)